

DELHI TECHNOLOGICAL UNIVERSITY

Python Programming

MC103



B. Tech. Mathematics and Computing

Submitted By:

Submitted To:

Experiment No. - 1

WAP that creates variables of numeric data types and perform arithmetic operations on them in Python.

Theory: -

Python supports different numeric data types such as int, float and complex. Arithmetic operators like +, -, *, /, //, %, and ** are used to perform calculations.

Code: -

```
a = int(input("Enter 1st number: "))
b = int(input("Enter 2nd number: "))

sum = a+b

sub = a-b

product = a*b

division = a/b

modulus = a%b

power = a**b

floor = a//b

print("Sum:",sum)

print("Subtraction:",sub)

print("Product:",product)

print("Division:",division)

print("Power:", power)

print("Floor Division:",floor)
```

Output: -

Enter 1st number: 23

Enter 2nd number: 4

Sum: 27

Subtraction: 19

Product: 92

Division: 5.75

Power: 279841

Floor Division: 5

Experiment No. - 2

Write a Python program to declare string variables and perform different operations on them.

Theory: -

A string is a sequence of characters written inside quotes. Python provides many built-in operations and functions to manipulate strings.

Code: -

```
str = "DTU is a prestigious college."
```

```
print(str + "Students from DTU contribute to society in various ways.")
```

```
print(str[4:13])
```

```
print("Title: ",str.title())
```

```
print("Upper: ",str.upper())
```

```
print("Lower: ",str.lower())
```

```
print("Reverse: ",str[::-1])
```

```
print("Length: ", len(str))
```

```
print("Repeat:", str*2)
```

```
print(str.split())
```

```
print(str.find('a'))
```

```
print(str.count('i'))
```

Output: -

DTU is a prestigious college.Students from DTU contibute to society in various ways.

is a pres

Title: Dtu Is A Prestigious College.

Upper: DTU IS A PRESTIGIOUS COLLEGE.

Lower: dtu is a prestigious college.

Reverse: .egelloC suoigitserp a si UTD

Length: 29

Repeat: DTU is a prestigious college.DTU is a prestigious college.

['DTU', 'is', 'a', 'prestigious', 'college.']

7

3

Experiment No. - 3

WAP to create, insert and delete Lists in Python.

Theory: -

A list is an ordered and mutable collection of values. Elements can be added, inserted, removed and modified easily.

Code: -

```
lang = ['Python', 'C', 'Java', 'Javascript', 'HTML']
```

```
print(lang)
```

```
lang.append('CSS')
```

```
print(lang)
```

```
lang.insert(2, 'Matlab')
```

```
print(lang)
```

```
x = lang.pop()
```

```
print(lang)
```

```
print(x)
```

```
lang.remove('C')
```

```
print(lang)
```

```
del lang[4]
```

```
print(lang)
```

Output: -

```
['Python', 'C', 'Java', 'Javascript', 'HTML']
```

```
['Python', 'C', 'Java', 'Javascript', 'HTML', 'CSS']
```

```
['Python', 'C', 'Matlab', 'Java', 'Javascript', 'HTML', 'CSS']
```

```
['Python', 'C', 'Matlab', 'Java', 'Javascript', 'HTML']
```

```
CSS
```

```
['Python', 'Matlab', 'Java', 'Javascript', 'HTML']
```

```
['Python', 'Matlab', 'Java', 'Javascript']
```

Experiment: - 4

WAP to create, insert and delete Dictionaries in Python.

Theory: -

A dictionary stores data in key-value form. Keys are unique and values can be updated or removed whenever required.

Code: -

```
student = {"Name": "Aditya", "Roll": "26/B4/019", "Branch": "MCE", "College": "DTU"}
```

```
print(student.items())
```

```
print(student.keys())
```

```
print(student.values())
```

```
student['City'] = "Delhi"
```

```
print(student)
```

```
student["Branch"] = "CSE"
```

```
print(student)
```

```
del student["College"]
```

```
print(student)
```

Output: -

```
dict_items([('Name', 'Aditya'), ('Roll', '26/B4/019'), ('Branch', 'MCE'), ('College', 'DTU')])
```

```
dict_keys(['Name', 'Roll', 'Branch', 'College'])
```

```
dict_values(['Aditya', '26/B4/019', 'MCE', 'DTU'])
```

```
{'Name': 'Aditya', 'Roll': '26/B4/019', 'Branch': 'MCE', 'College': 'DTU', 'City': 'Delhi'}
```

```
{'Name': 'Aditya', 'Roll': '26/B4/019', 'Branch': 'CSE', 'College': 'DTU', 'City': 'Delhi'}
```

```
{'Name': 'Aditya', 'Roll': '26/B4/019', 'Branch': 'CSE', 'City': 'Delhi'}
```

Experiment: - 5

Write a Python program to create, insert and delete Sets and Tuples in Python.

Theory: -

A set stores unique unordered values and can be modified. A tuple stores ordered values but cannot be changed after creation.

Code:-

```
colors = {"Red", "Blue", "Green"}
```

```
print("Original Set =", colors)
```

```
colors.add("Yellow")
```

```
print("After Insert =", colors)
```

```
colors.remove("Blue")
```

```
print("After Delete =", colors)
```

```
numbers = (10, 20, 30, 40)
```

```
print("Original Tuple =", numbers)
```

```
new_tuple = numbers + (50,)
```

```
print("After Insert =", new_tuple)
```

```
new_tuple = new_tuple[:2] + new_tuple[3:]
```

```
print("After Delete =", new_tuple)
```

Output: -

Original Set = {'Blue', 'Green', 'Red'}

After Insert = {'Yellow', 'Blue', 'Green', 'Red'}

After Delete = {'Yellow', 'Green', 'Red'}

Original Tuple = (10, 20, 30, 40)

After Insert = (10, 20, 30, 40, 50)

After Delete = (10, 20, 40, 50)

Experiment: - 6

Write a Python program to print Fibonacci series using for loop.

Theory: -

The Fibonacci series is a sequence in which each number is the sum of the previous two numbers. It begins with 0 and 1.

Code: -

```
n = 10
a = 0
b = 1
print(a, end = " ")
print(b, end = " ")

for i in range(n):
    temp = b
    b = a + b
    a = temp
    print(b, end = " ")
```

Output:-

0 1 1 2 3 5 8 13 21 34 55 89

Experiment: - 7

Create a function `search_list` which takes a list of values as arguments and searches a particular number in the list.

Theory: -

A function is a reusable block of code. Linear search checks each element of the list one by one until the required value is found.

Code: -

```
def search_list(values, key):  
    for i in range(len(values)):  
        if values[i] == key:  
            return i  
    return -1  
  
numbers = [12, 25, 7, 40, 18, 30]  
key = 18  
position = search_list(numbers, key)  
print("List =", numbers)  
print("Number to search =", key)  
if position != -1:  
    print("Number found at index", position)  
else:  
    print("Number not found")
```

Output: -

List = [12, 25, 7, 40, 18, 30]

Number to search = 18

Number found at index 4